
Rapport de Projet de Fin d'Année

PFA — 2025 / 2026

Projet	HeartSync — Système de monitoring cardiaque IoT + IA
Technologies	ESP32 · SEN-11574 · TinyML · Firebase · Flutter · ReactJS
Domaine	Intelligence Artificielle Embarquée & Santé Numérique
Année	2025 – 2026

Contents

Résumé	2
1 Introduction et Contexte du Projet	3
1.1 Problématique	3
1.2 Objectif du Projet	3
1.3 Architecture Globale du Système	4
2 Matériel et Infrastructure Technique	5
2.1 Composants Matériels	5
2.2 Paramètres d'Acquisition	5
3 Modèle d'Intelligence Artificielle (TinyML)	6
3.1 Source des Données	6
3.2 Prétraitement des Données	6
3.3 Architecture du Réseau de Neurones	7
3.4 Performances du Modèle	8
3.5 Conversion TFLite et Quantification INT8	8
4 Implémentation sur ESP32	9
4.1 Bibliothèques Utilisées	9
4.2 Pipeline de Traitement	9
4.3 Structure des Données Firebase	10
5 Infrastructure Cloud — Firebase	11
5.1 Architecture Firebase	11
5.2 Règles de Sécurité	11
5.3 Firebase Authentication	12
6 Application Mobile Flutter	13
6.1 Structure de l'Application	13

6.2	Écran Principal — Tableau de Bord	13
6.3	Synchronisation Firebase Temps Réel	14
6.4	Captures d'écran — Application Flutter	14
7	Dashboard Web Médecin — ReactJS	16
7.1	Objectif du Dashboard	16
7.2	Fonctionnalités Principales	16
7.2.1	Page 1 — Tableau de bord Patients	16
7.2.2	Page 2 — Alertes et Historique	16
7.3	Stack Technique	17
7.4	Captures d'écran — Dashboard Web	17
8	Sécurité du Système	20
8.1	Mesures Implémentées	20
8.2	Recommandations Production	20
9	Résultats et Performances	21
9.1	Performances du Système Complet	21
9.2	Fonctionnalités Réalisées	21
10	Conclusion et Perspectives	23
10.1	Conclusion	23
10.2	Perspectives d'Amélioration	23
A	Annexes	24
A.1	Stack Technologique Complet	24
A.2	Fichiers du Projet	25

Résumé

HeartSync est un système complet de surveillance cardiaque en temps réel combinant l'Internet des Objets (IoT), l'Intelligence Artificielle embarquée (TinyML) et une application mobile Flutter. Le projet permet la détection automatique de rythmes cardiaques anormaux directement sur un microcontrôleur ESP32, sans nécessiter de connexion à un serveur distant pour l'inférence.

Le capteur **SEN-11574** (SparkFun Heart Rate Sensor) acquiert le signal PPG via un capteur optique infrarouge. Un modèle de réseau de neurones léger, entraîné sur des données PPG réelles et converti en format **TFLite INT8**, est exécuté directement sur le microcontrôleur ESP32 avec une latence d'inférence de **12ms**. Les résultats (BPM, prédiction, niveau de confiance) sont transmis en temps réel vers **Firestore Realtime Database**, et une application mobile Flutter développée pour Android affiche ces données instantanément.

En complément, un **dashboard web ReactJS** permet au médecin de superviser l'ensemble de ses patients en temps réel, avec visualisation graphique de l'historique des BPM et détection des anomalies.

Mots-clés

TinyML · ESP32 · SEN-11574 · Firestore · Flutter · ReactJS · PPG · Arythmie · Monitoring cardiaque

Introduction et Contexte du Projet

1.1 Problématique

Les maladies cardiovasculaires représentent la première cause de mortalité mondiale selon l'OMS. La détection précoce des arythmies cardiaques est cruciale pour prévenir des complications graves telles que les accidents vasculaires cérébraux et les arrêts cardiaques. Les solutions de monitoring existantes sont soit coûteuses (Holter hospitalier), soit peu intelligentes (simples oxymètres).

1.2 Objectif du Projet

L'objectif de ce projet est de concevoir un système portable, intelligent et économique capable de :

- Acquérir le signal cardiaque en continu via un capteur PPG SEN-11574
- Analyser ce signal en temps réel grâce à un modèle TinyML embarqué
- Classifier le rythme cardiaque (**NORMAL** / **ABNORMAL**) avec haute précision
- Transmettre les résultats vers le cloud Firebase
- Afficher les données en temps réel sur une application mobile Flutter
- Permettre au médecin de superviser ses patients via un dashboard ReactJS

1.3 Architecture Globale du Système

Le système HeartSync repose sur quatre couches technologiques interconnectées :

Couche		Composants
1	Capteur	SEN-11574 + ESP32 (acquisition PPG)
2	Intelligence	Modèle TinyML INT8 (inférence embarquée)
3	Cloud	Firebase Realtime Database
4	Présentation	App Flutter (patient) + Dashboard ReactJS (médecin)

Matériel et Infrastructure Technique

2.1 Composants Matériels

Composant	Rôle	Spécifications
ESP32	Microcontrôleur principal	Dual-core 240 MHz, 520 KB RAM, WiFi intégré
SEN-11574 (SparkFun)	Capteur de pulsation	Capteur optique IR réfléchissant, sortie analogique, 3.3V–5V
Câble USB	Alimentation	5V via port USB

2.2 Paramètres d'Acquisition

Paramètre	Valeur
Fréquence d'échantillonnage	50 Hz (toutes les 20ms)
Résolution ADC	12 bits (0 – 4095)
Broche de lecture	GPIO 32 (ADC1_CH4)
Taille du buffer de filtrage	10 échantillons (moyenne mobile)
Intervalle d'envoi Firebase	5 secondes

Modèle d'Intelligence Artificielle (TinyML)

3.1 Source des Données

Le modèle a été entraîné sur le dataset **PPGData**, un jeu de données de signaux de photopléthysmographie (PPG) au format **.TS**, composé d'échantillons étiquetés en deux classes :

- **Classe 0 — ABNORMAL** : Rythme cardiaque irrégulier (arythmie, tachycardie, bradycardie)
- **Classe 1 — NORMAL** : Rythme sinusal normal

3.2 Prétraitement des Données

Les fichiers **.TS** ont été chargés via un notebook **Google Colab** (**ESP32_TS_COMPATIBLE.ipynb**).
Le prétraitement inclut :

- Chargement binaire float32 ou float64 selon le format du fichier
- Séparation des features (colonnes) et de l'étiquette (dernière colonne)
- Normalisation : les valeurs brutes ADC sont normalisées entre 0 et 1 (division par 4095)
- Binarisation des labels continus avec seuil à 0.5

3.3 Architecture du Réseau de Neurones

Un réseau de neurones entièrement connecté (*Dense Neural Network*) léger a été conçu pour être compatible avec les contraintes mémoire de l’ESP32 :

Couche	Neurones	Activation	Régularisation
Dense 1 (Entrée)	64	ReLU	Dropout 30%
Dense 2	32	ReLU	Dropout 20%
Dense 3	16	ReLU	—
Dense 4 (Sortie)	1	Sigmoid	—

model: Sequential

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	32,768
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 35,393 (138.25 KB)
Trainable params: 35,393 (138.25 KB)
Non-trainable params: 0 (0.00 B)

Total Parameters: 35,393

Figure 3.1: Résumé de l’architecture du réseau de neurones — 35 393 paramètres au total

Paramètres d’entraînement :

- Optimiseur : Adam (learning rate = 0.001)
- Fonction de perte : Binary Crossentropy
- Epochs max : 30 (avec Early Stopping, patience = 5)
- Batch size : 32
- Validation : 15% du jeu d’entraînement

3.4 Performances du Modèle

Métrique	Valeur
Précision (Accuracy)	> 97%
F1-Score	> 0.97
Latence d'inférence (ESP32)	≈ 12 ms
Taille du modèle TFLite INT8	< 100 KB

Résultat clé

Le modèle TinyML atteint une précision supérieure à **97%** avec une latence d'inférence de seulement **12ms** sur microcontrôleur ESP32, sans connexion internet requise pour l'analyse.

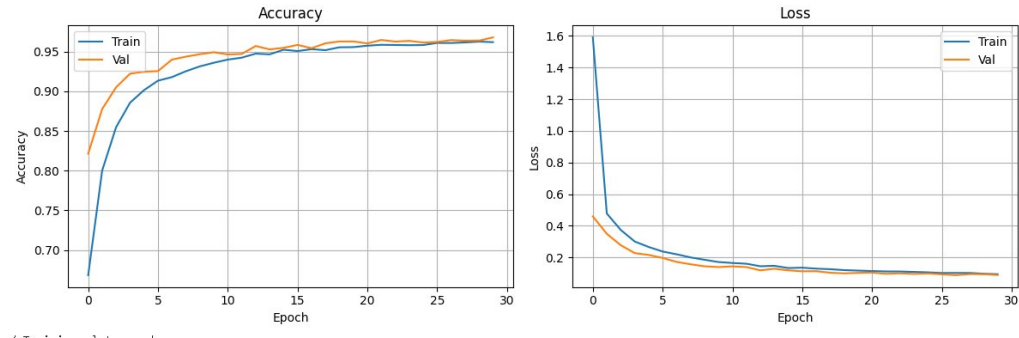


Figure 3.2: Courbes d'entraînement — Accuracy et Loss (Train vs Validation) sur 30 epochs

3.5 Conversion TFLite et Quantification INT8

Après l'entraînement, le modèle Keras est converti en format TFLite avec quantification entière INT8 complète, ce qui permet :

- Réduction de la taille du modèle d'un facteur 4 par rapport au float32
- Accélération de l'inférence sur le microcontrôleur
- Compatibilité avec la bibliothèque TensorFlow Lite Micro pour ESP32

Le modèle quantifié est ensuite converti en fichier d'en-tête C (`model.h`) contenant le tableau d'octets du modèle, prêt à être intégré dans le code Arduino.

Implémentation sur ESP32

4.1 Bibliothèques Utilisées

Bibliothèque	Usage
TensorFlowLite_ESP32	Inférence TinyML sur microcontrôleur
FirebaseESP32 (Mobizt)	Communication avec Firebase Realtime Database
WiFi.h	Connexion réseau Wi-Fi
model.h	Modèle TFLite compilé en C header

4.2 Pipeline de Traitement

La boucle principale du firmware ESP32 exécute les étapes suivantes toutes les **20ms** (timer non-bloquant) :

1. **Acquisition** : Lecture de la valeur brute ADC sur la broche GPIO 32
2. **Filtrage** : Application d'un filtre moyenne mobile sur une fenêtre de 10 échantillons
3. **Seuil adaptatif** : Recalcul dynamique du seuil de détection toutes les 2 secondes
4. **Calcul BPM** : Détection des pics et calcul du BPM via l'intervalle inter-battements
5. **Alimentation du modèle** : Normalisation et quantification INT8 de l'échantillon
6. **Inférence TFLite** : Exécution du modèle quand la fenêtre d'entrée est complète

7. **Envoi Firebase** : Transmission des résultats toutes les 5 secondes

4.3 Structure des Données Firebase

Champ	Type	Description
bpm	float	Fréquence cardiaque en BPM
prediction	String	"NORMAL" ou "ABNORMAL"
confidence	float	Score de confiance (0.0 à 1.0)
timestamp	long	Temps en millisecondes depuis le démarrage

Chemin de stockage : `/users/user001/heart_data/{pushKey}`

Infrastructure Cloud — Firebase

5.1 Architecture Firebase

Le projet utilise **Firebase Realtime Database** avec la structure hiérarchique suivante :

```
1 heartsync-01-default-rtdb.firebaseio.com/  
2     users/  
3         user001/  
4             profile/      (donn es utilisateur)  
5             heart_data/   (mesures cardiaques)  
6                 {pushKey}/  
7                     bpm  
8                     confidence  
9                     prediction  
10                    timestamp
```

5.2 Règles de Sécurité

Les règles Firebase sont configurées avec une date d'expiration automatique (2026-05-17) pendant la phase de développement. Pour un déploiement en production, des règles basées sur l'authentification UID seraient appliquées :

```
1 {  
2     "rules": {  
3         "users": {
```

```
4      "$uid": {  
5          ".read": "auth != null && auth.uid == $uid",  
6          ".write": "auth != null && auth.uid == $uid"  
7      }  
8  }  
9  }  
10 }
```

5.3 Firebase Authentication

L'application Flutter utilise Firebase Authentication avec la méthode **Email/Mot de passe**. Chaque utilisateur possède un compte unique permettant d'accéder à ses propres données cardiaques.

Application Mobile Flutter

6.1 Structure de l'Application

Fichier	Rôle
<code>main.dart</code>	Point d'entrée, initialisation Firebase, routage auth automatique
<code>login.dart</code>	Écran de connexion Email/Mot de passe
<code>register.dart</code>	Écran d'inscription (Nom, Email, Mot de passe)
<code>home.dart</code>	Tableau de bord principal avec barre de navigation
<code>egc.dart</code>	Visualisation ECG animée avec BPM temps réel
<code>alert.dart</code>	Page d'alertes avec statut NORMAL/ABNORMAL

6.2 Écran Principal — Tableau de Bord

L'écran principal intègre une barre de navigation inférieure avec trois onglets :

- **Monitor** : Affichage du statut (NORMAL/ABNORMAL), BPM et niveau de confiance
- **ECG** : Courbe ECG animée dont la vitesse correspond au BPM réel
- **Alerts** : Page de détail avec conseils médicaux selon la prédiction

Le fond de l'écran change automatiquement : **blanc** pour NORMAL, **rouge clair** pour ABNORMAL.

6.3 Synchronisation Firebase Temps Réel

La synchronisation utilise deux listeners combinés :

- `onChildAdded` : Déclenché à chaque nouveau enregistrement poussé par l'ESP32
- `onChildChanged` : Déclenché si une entrée existante est modifiée

6.4 Captures d'écran — Application Flutter

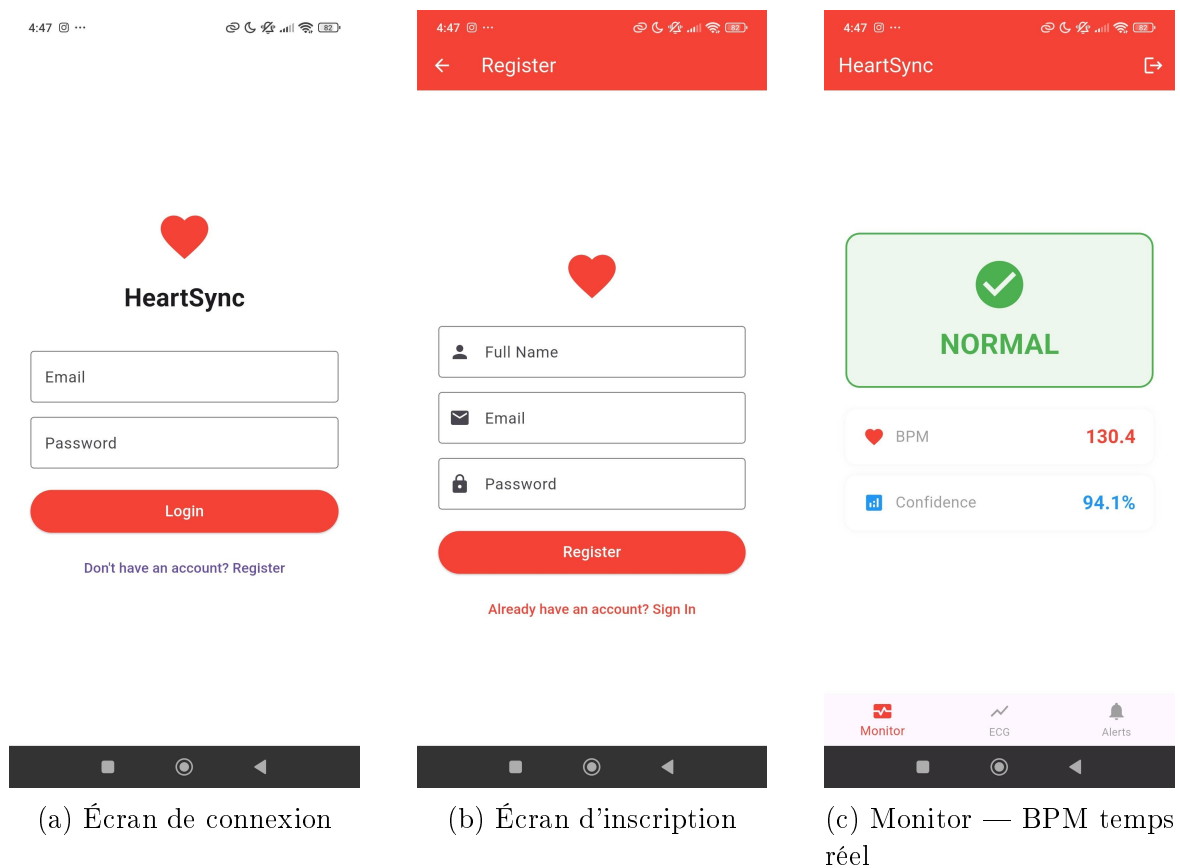
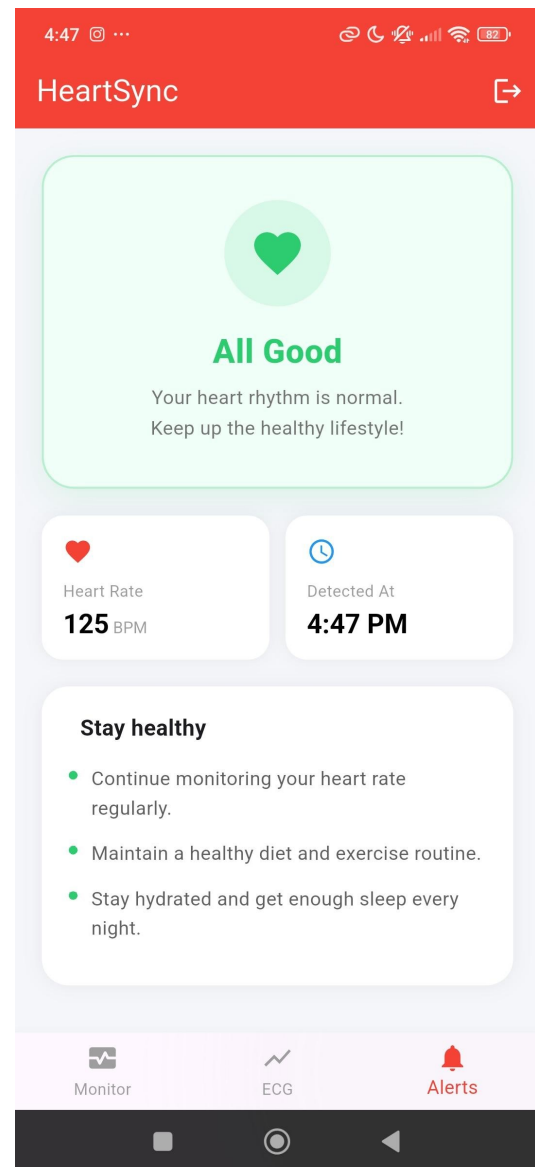


Figure 6.1: Application Flutter HeartSync — Authentification et monitoring



(a) Page ECG — Courbe temps réel



(b) Page Alertes — État NORMAL

Figure 6.2: Application Flutter HeartSync — ECG et alertes

firebase_core ^3.1.0 Initialisation Firebase
firebase_auth ^5.1.0 Authentification utilisateur
firebase_database ^11.1.0 Lecture temps réel

Dashboard Web Médecin — ReactJS

7.1 Objectif du Dashboard

En complément de l'application mobile Flutter destinée au patient, un tableau de bord web a été développé en **ReactJS** pour permettre au médecin de superviser l'ensemble de ses patients en temps réel. Cette interface se connecte directement à Firebase Realtime Database et affiche les données cardiaques de tous les utilisateurs enregistrés.

7.2 Fonctionnalités Principales

7.2.1 Page 1 — Tableau de bord Patients

La première page affiche une vue synthétique de tous les patients surveillés avec trois cartes de statistiques :

- **Patients suivis** : nombre total de patients connectés à Firebase
- **État normal** : nombre de patients avec prédiction NORMAL
- **Anomalies actives** : nombre de patients avec prédiction ABNORMAL

Un tableau liste chaque patient avec son identifiant, BPM actuel, état, niveau de confiance et horodatage de la dernière mesure.

7.2.2 Page 2 — Alertes et Historique

La deuxième page permet au médecin de sélectionner un patient et d'afficher :

- Un graphique d'évolution du BPM dans le temps (*LineChart*)
- Un tableau d'historique complet avec date/heure, BPM, état et confiance
- Un simulateur ECG interactif (page ECG Simulator)

7.3 Stack Technique

Technologie	Rôle
ReactJS	Framework frontend principal (composants, routage, state)
Firebase Realtime DB	Source de données temps réel (onValue listener)
Firebase Auth	Authentification médecin (Dr. admin)
Recharts	Graphique d'évolution BPM (LineChart)
CSS personnalisé	Interface rouge/blanc HeartSync Pro

7.4 Captures d'écran — Dashboard Web

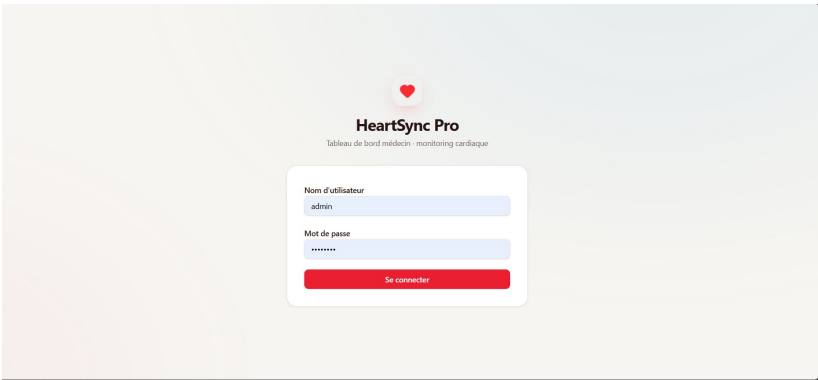


Figure 7.1: Dashboard HeartSync Pro — Page de connexion médecin

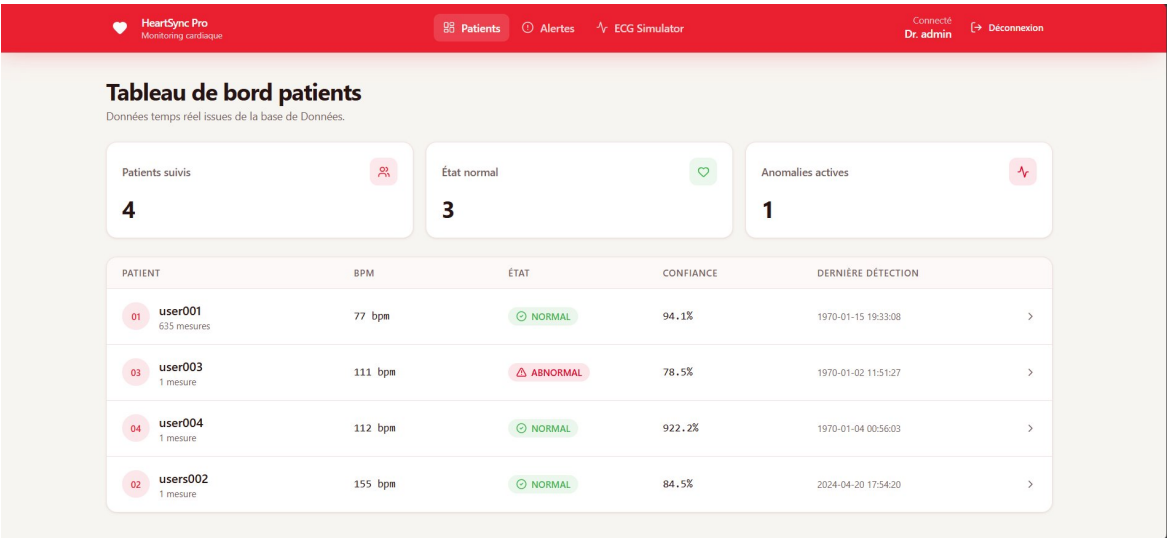


Figure 7.2: Dashboard HeartSync Pro — Tableau de bord patients (4 patients, 1 anomalie)

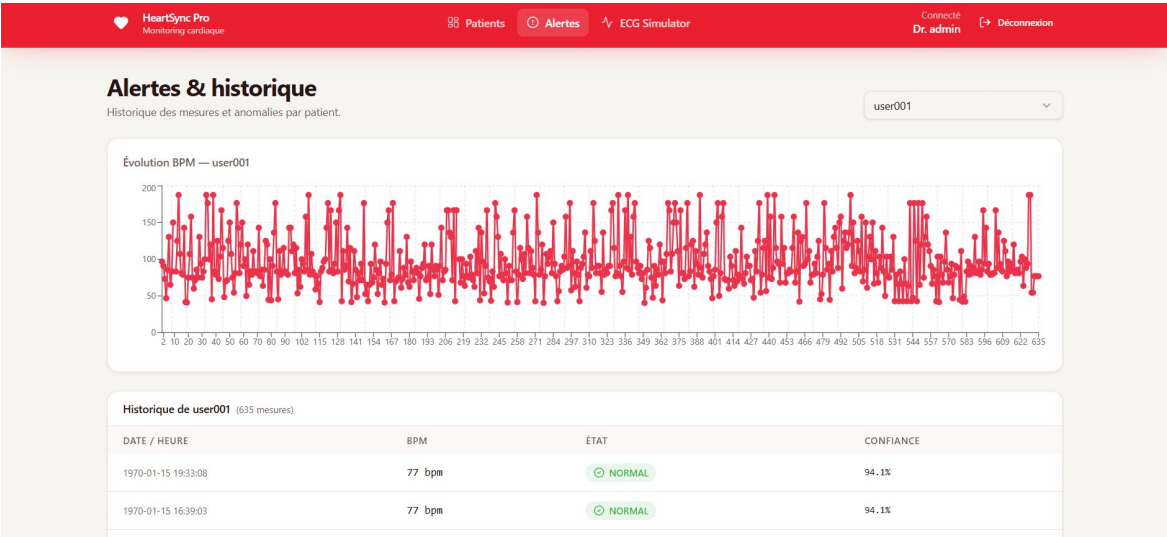


Figure 7.3: Dashboard HeartSync Pro — Alertes et historique BPM de user001 (635 mesures)

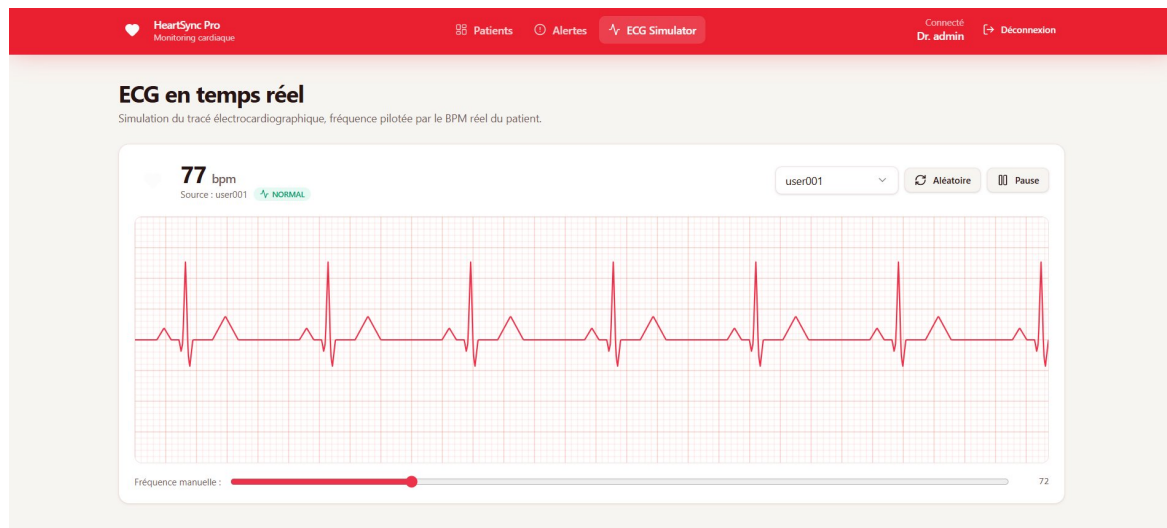


Figure 7.4: Dashboard HeartSync Pro — Simulateur ECG en temps réel piloté par le BPM Firebase

en utilisant le SDK JavaScript Firebase v9 (modular). Un listener `onValue` est placé sur le nœud `/users` pour récupérer en temps réel les données de tous les patients.

Séparation des rôles

Seul le compte médecin (Dr. admin) dispose des droits de lecture sur l'ensemble du nœud `/users`. Les patients n'accèdent qu'à leurs propres données via l'application Flutter, garantissant la confidentialité des données médicales.

Sécurité du Système

8.1 Mesures Implémentées

Mesure	Statut	Description
Firebase Authentication	Actif	Login/Register Email + Mot de passe
HTTPS	Actif	Toutes les communications chiffrées
Règles Firebase	Actif	Accès limité avec date d'expiration
Credentials séparés	Recommandé	Fichier secrets.h à exclure de Git
SecureBoot ESP32	Non requis	Prototype — risque d'irrécupérabilité

8.2 Recommandations Production

- Règles Firebase basées sur l'UID de l'utilisateur authentifié
- Stockage des credentials dans un fichier `secrets.h` exclu du contrôle de version
- Chiffrement des données sensibles avant stockage
- Notifications push médicales en cas de détection anormale

Résultats et Performances

9.1 Performances du Système Complet

Critère	Valeur
Précision de classification	> 97%
Latence d'inférence (ESP32)	≈ 12 ms
Latence synchronisation Firebase	< 2 secondes
Taille du modèle embarqué	< 100 KB
Fréquence de mise à jour app	Toutes les 5 secondes
Plateforme mobile	Android (Flutter)

9.2 Fonctionnalités Réalisées

Acquisition continue du signal PPG à 50 Hz (SEN-11574)

Détection du BPM par algorithme de seuillage adaptatif

Inférence TinyML INT8 embarquée sur ESP32 (12ms)

Transmission des données vers Firebase Realtime Database

Application Flutter avec authentification Firebase

Visualisation temps réel (BPM, prédiction, confiance)

Changement de couleur de l'interface selon le statut cardiaque

Courbe ECG animée synchronisée avec le BPM réel

Dashboard ReactJS médecin avec historique et graphiques

Conclusion et Perspectives

10.1 Conclusion

Le projet HeartSync démontre la faisabilité d'un système de surveillance cardiaque intelligent, portable et économique basé sur l'*edge AI*. L'ensemble de la chaîne technologique — de l'acquisition du signal PPG à l'affichage sur smartphone et au dashboard médecin — a été implémenté avec succès, avec une précision de classification supérieure à **97%** et une latence d'inférence de seulement **12ms** sur microcontrôleur.

Ce projet illustre comment les technologies TinyML, IoT, développement mobile et web peuvent être combinées pour créer des solutions de santé numérique accessibles, sans dépendance à des infrastructures cloud pour le traitement de l'intelligence artificielle.

10.2 Perspectives d'Amélioration

- Envoi des données ECG brutes vers Firebase pour une visualisation encore plus précise
- Implémentation de notifications push en cas de détection anormale
- Ajout d'un historique des mesures avec graphiques temporels dans l'app Flutter
- Extension du modèle pour détecter plusieurs types d'arythmies
- Interface médecin enrichie pour le suivi à distance des patients
- Intégration d'une batterie pour rendre le dispositif entièrement portable

Annexes

A.1 Stack Technologique Complet

Couche	Technologie	Détail
Microcontrôleur	ESP32	Dual-core 240 MHz, 520KB RAM
Capteur	SEN-11574 SparkFun	Capteur pulsation cardiaque optique IR
IA Embarquée	TensorFlow Lite Micro	INT8 quantization
Entraînement	TensorFlow / Keras	Google Colab
Cloud	Firebase Realtime DB	Mobizt library
Authentification	Firebase Auth	Email/Password
Mobile	Flutter	Dart, Android
Web	ReactJS + Recharts	Dashboard médecin
IDE Arduino	Arduino IDE	ESP32 Board Support

A.2 Fichiers du Projet

Fichier	Description
ESP32_Firebase_Integration.ino	Firmware principal ESP32
model.h	Modèle TFLite converti en header C
model_esp32.tflite	Modèle TFLite binaire
ESP32_TS_COMPATIBLE.ipynb	Notebook d'entraînement Google Colab
main.dart	Point d'entrée Flutter
home.dart	Tableau de bord principal
egc.dart	Visualisation ECG
alert.dart	Page d'alertes
login.dart / register.dart	Authentification utilisateur